

Full Stack Development with MERN

Project Documentation

1. Introduction

Project Title

HouseHunt – House Rental and Property Management System

Team Members

Team Member	Role
Machanuru Rajesh Jyoshna	Project Architecture Desinging & Documentation
Lokeshwar Naik Vadithya	Project Execution
Masthan Manikala	Database Developer & Project Configuration & Setup
Rishitha Depuru	Full Stack Development & Testing
Sireesha K	Project Architecture Desinging & Documentation

2. Project Overview

Purpose

HouseHunt is a **full-stack web application** developed using the **MERN Stack** that simplifies the process of **house renting and property management**. The platform connects **Tenants**, **Landlords**, and **Administrators** through a single digital marketplace. It enables users to

search rental properties, manage listings, book property visits, and monitor rental activities through secure role-based dashboards.

Features

- Role-Based Access (**User, Landlord, Admin**)
- Secure User Registration & Login
- JWT Authentication
- Property Search with Advanced Filters
- Property Listing Management
- Wishlist Management
- Property Visit Booking
- Booking History
- Property Image Upload
- Property Approval System
- User Dashboard
- Landlord Dashboard
- Admin Dashboard
- Property Reviews
- Responsive User Interface
- Dark Mode Support
- MongoDB Database Integration

3. Architecture

Frontend

The frontend is developed using **React.js** and **Vite**, following a **component-based architecture**. React Router is used for navigation, while Context API manages **Authentication, Theme, and Role** states. The interface is responsive and optimized for desktop and mobile devices.

Backend

The backend is developed using **Node.js** and **Express.js**, following the **REST API architecture**. Business logic is implemented using **Controllers**, while API requests are managed through **Routes**. **JWT Authentication, Role-Based Authorization, and Multer** are used to provide secure authentication and image upload functionality.

Database

The application uses **MongoDB** with **Mongoose ODM** for data storage. Collections include:

- Users
- Properties
- Bookings
- Reviews
- Wishlist

Relationships between collections are maintained using **Object References (ObjectId)** to ensure efficient querying and data consistency.

4. Setup Instructions

Prerequisites

Install the following software before running the project:

- Node.js
- npm
- MongoDB Community Server
- Visual Studio Code
- Git

- **Installation**

Clone Repository

```
git clone https://github.com/your-repository/HouseHunt.git
```

Install Dependencies

```
npm run install-all
```

Configure Environment Variables

Create a **backend/.env** file.

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/househunt
JWT_SECRET=your_secret_key
JWT_EXPIRES_IN=7d
FRONTEND_URL=http://localhost:5173
```

Seed Database

npm run seed

5. Folder Structure

Client (Frontend)

The **frontend** is organized into reusable **components**, **contexts**, **services**, **routes**, and **modules** for **Users**, **Landlords**, and **Administrators**. It follows a modular architecture to improve maintainability and scalability.

Server (Backend)

The **backend** is divided into **Controllers**, **Models**, **Routes**, **Middlewares**, **Configuration Files**, and **Uploads**. It follows the **MVC architecture**, ensuring separation of business logic, routing, and database operations.

6. Running the Application

Install Dependencies

npm run install-all

Start MongoDB

mongod

Seed Database

npm run seed

Start Application

npm run dev

OR

Frontend

cd frontend

npm run dev

Backend

```
cd backend
npm run dev
```

Frontend URL

<http://localhost:5173>

Backend URL

<http://localhost:5000/api>

7. API Documentation

The backend exposes **RESTful APIs** for authentication, property management, bookings, reviews, wishlist, and administration.

Module	Method	Endpoint
Authentication	POST	/api/auth/register
Authentication	POST	/api/auth/login
Authentication	POST	/api/auth/forgot-password
Users	GET	/api/users/profile
Users	PUT	/api/users/profile
Properties	GET	/api/properties
Properties	POST	/api/properties
Properties	PUT	/api/properties/:id
Properties	DELETE	/api/properties/:id
Bookings	POST	/api/bookings
Bookings	GET	/api/bookings/tenant
Reviews	POST	/api/reviews
Wishlist	GET	/api/wishlist
Wishlist	POST	/api/wishlist/toggle
Admin	GET	/api/admin/analytics
Admin	GET	/api/admin/users

Admin PUT /api/admin/users/:id/role

8. Authentication

The HouseHunt application implements **JWT (JSON Web Token)** based authentication. After successful login, the server generates a **JWT Token**, which is stored on the client side. Protected APIs are secured using **Authentication Middleware**, while **Role-Based Authorization** ensures that only **Users**, **Landlords**, or **Administrators** can access their respective dashboards and functionalities.

9. User Interface

The application provides a modern and responsive user interface consisting of:

- Role Selection Page
- Login & Registration Pages
- Property Search Page
- User Dashboard
- Wishlist
- Booking History
- Landlord Dashboard
- Property Management
- Admin Dashboard
- Property Approval Interface

(Insert screenshots with descriptions here.)

10. Testing

The HouseHunt application was tested using **Manual Functional Testing** and **API Testing**.

Testing Tools

- Postman
- Chrome Developer Tools
- MongoDB Compass

Tested Modules

- User Authentication
- Property Management
- Property Booking
- Wishlist

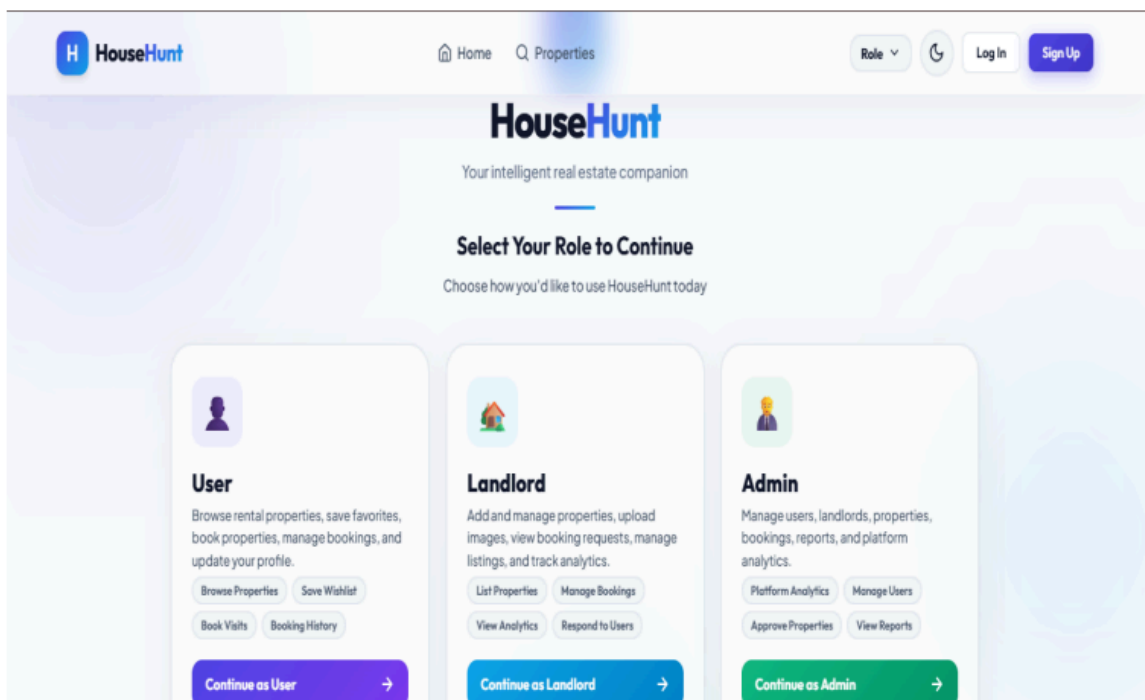
- Reviews
- Admin Operations
- Role-Based Authorization

All core functionalities were verified to ensure proper system behavior and data consistency.

11. Screenshots or Demo

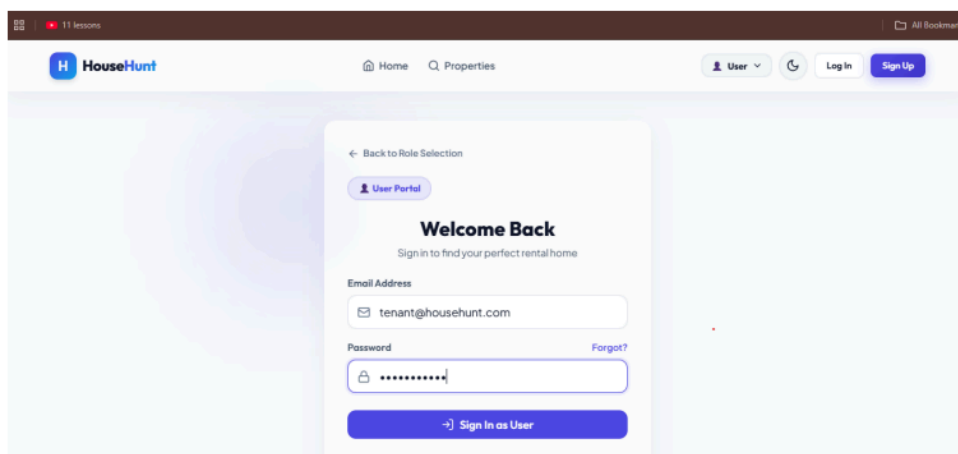
The project includes screenshots demonstrating:

- **Role Selection**



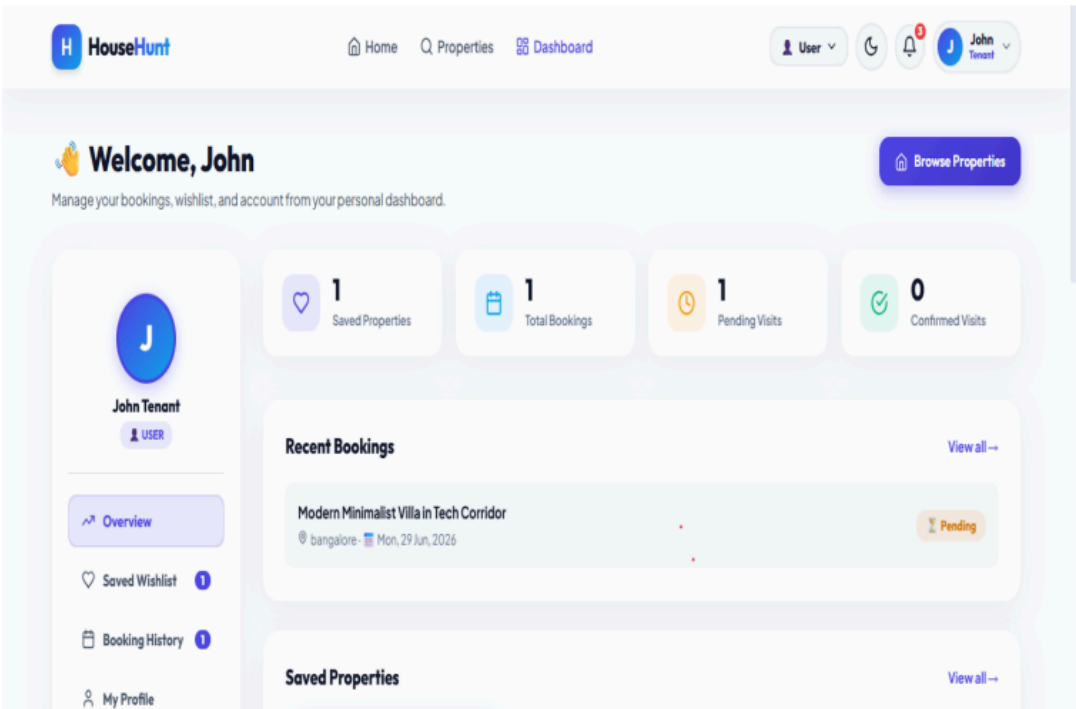
The **HouseHunt Role Selection Page** is the first screen displayed to users, allowing them to choose between **User**, **Landlord**, and **Admin** roles. Each role provides a brief overview of its functionalities and a **"Continue"** button for role-specific login and dashboard access, ensuring a secure and personalized user experience.

- **User Login**



User Login Page: The **User Login Interface** enables registered users to securely access

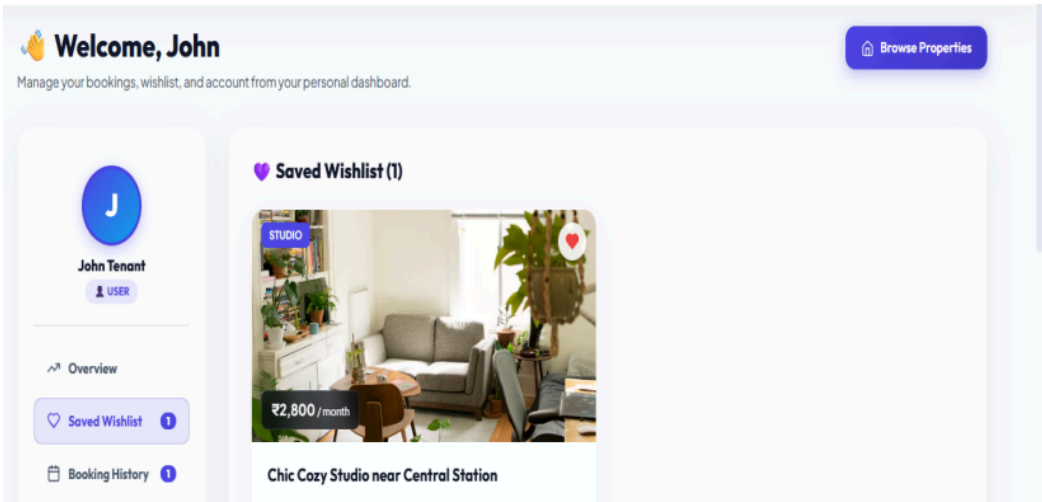
- **User Dashboard**



The **User Profile Dashboard** provides a personalized overview of the user's account, displaying **saved properties**, **booking statistics**, **pending visits**, and **confirmed visits**. It also offers quick navigation to **Wishlist**, **Booking History**, and **Profile Management**, enabling users to efficiently manage their rental activities from a single dashboard.

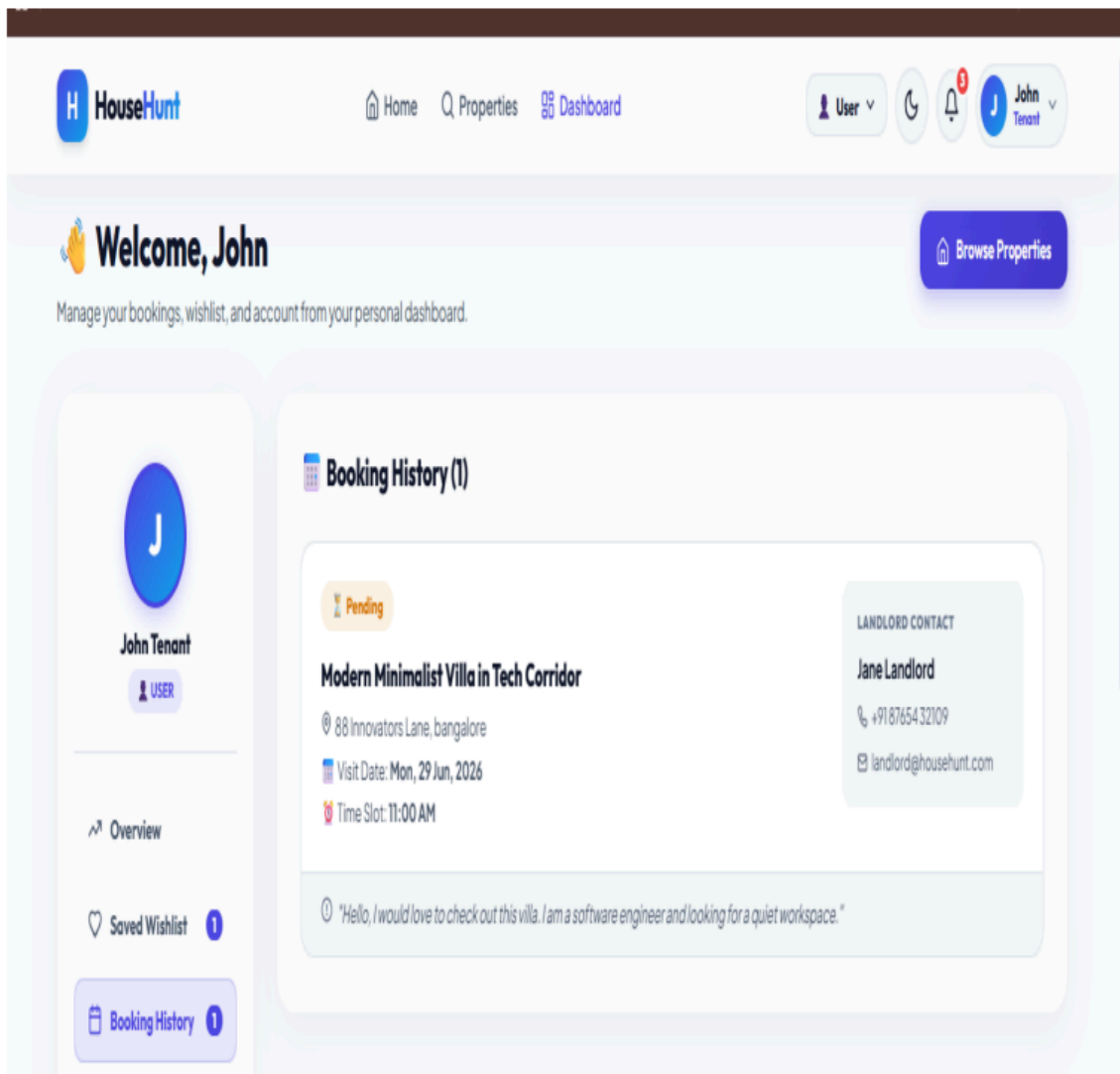
- **Wishlist**

The **Saved Wishlist** allows users to view and manage their **favorite properties** in one



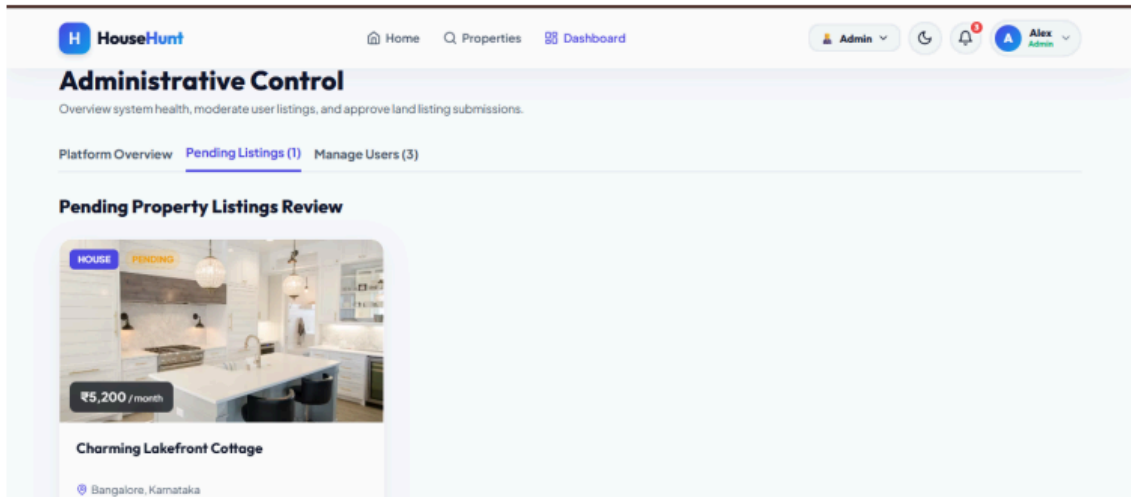
place. Each listing displays essential details such as the **property image**, **type**, **rental price**, and **location**, enabling users to quickly access and compare their preferred properties

- **Booking History**

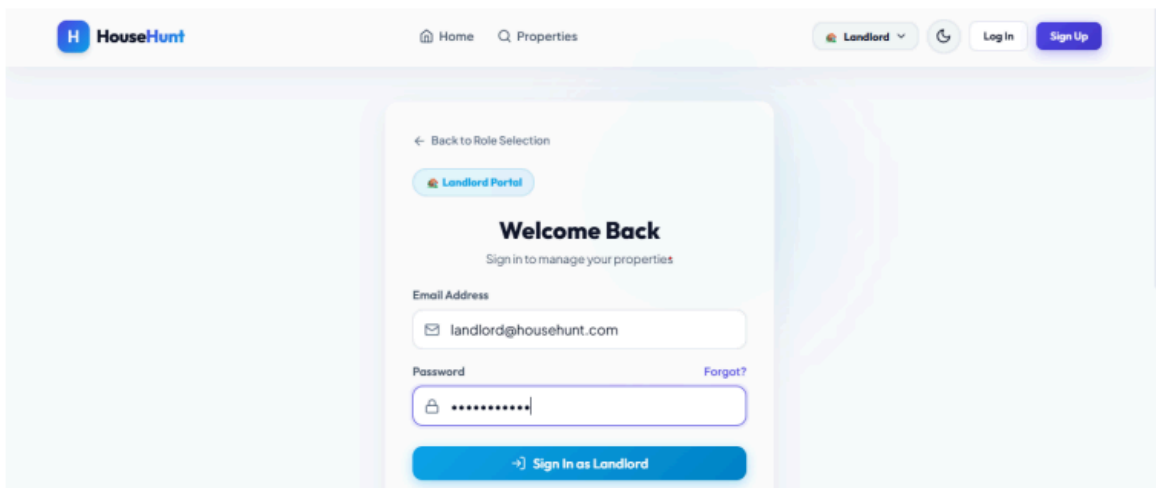


The **Booking History** section enables users to track all their **property visit bookings** along with their **booking status**, **scheduled date**, and **time slot**. It also displays the **landlord's contact information** and the user's booking message, allowing easy communication and efficient management of visit requests.

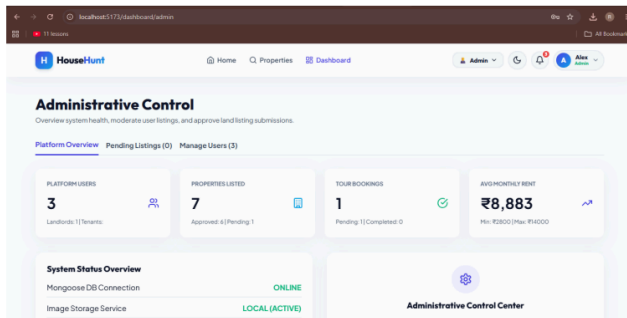
- **Property Search**



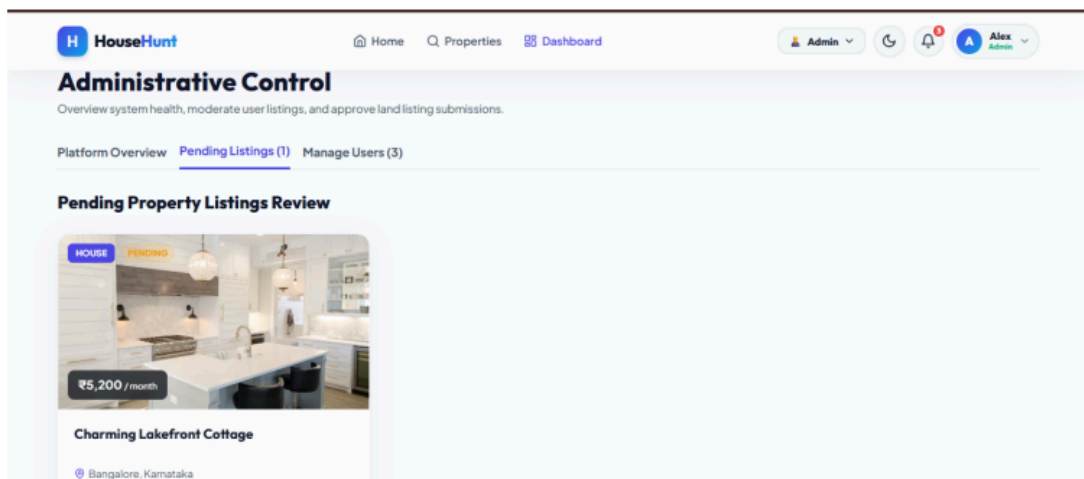
- **Landlord Dashboard**



- **Admin Dashboard**



● Pending Property Approval



A demo video can also be included to showcase the complete workflow of the application.

12. Known Issues

- Email notifications are currently simulated.
- Online payment integration is not implemented.
- Real-time notifications are unavailable.
- Internet connectivity is required for accessing property images.

- Cloud deployment is not included in the current version.

13. Future Enhancements

The following enhancements are planned for future versions:

- Online Rent Payment Gateway
- AI-Based Property Recommendations
- Google Maps Integration
- Live Chat Between Tenant and Landlord
- Email Verification
- PDF Rental Agreement Generation
- Real-Time Notifications
- Mobile Application Development
- Cloud Deployment (AWS/Azure)
- Advanced Analytics Dashboard